MODULE 6

Lab 1: Scaffold & First Tests with Claude Code

Duration	Format	Audience	Day / Position
45 min	**Hands-On Lab**	**Mixed Engineers**	**Day 1 / Sixth**

LAB OBJECTIVES

By the end of this lab, participants will have:

1. Initialized a new Python project using /init and reviewed the auto-generated CLAUDE.md file
2. Customized CLAUDE.md with TI-specific coding standards, project metadata, and team conventions
3. Scaffolded a complete Python project structure including venv/uv, ruff, black, pytest, and pre-commit configuration
4. Generated core application modules from natural language descriptions using Claude Code prompts
5. Written and executed initial unit tests via Claude Code with coverage reporting
6. Committed all work to git with meaningful commit messages generated by Claude Code

🛠 LAB PREREQUISITES

- Claude Code installed and authenticated (verified in Module 1)
- Terminal open in the course repo directory: cd ~/ti-vibe-coding
- Python 3.10+ available (python3 --version)
- Git configured (git config --global user.name and user.email set)

LAB TIMING BREAKDOWN

0:00 – 0:05	**Exercise 1: Project Initialization with /init**
0:05 – 0:12	**Exercise 2: Customize CLAUDE.md for Your Team**
0:12 – 0:22	**Exercise 3: Scaffold Python Project Structure**
0:22 – 0:32	**Exercise 4: Generate Core Modules with Claude Code**
0:32 – 0:42	**Exercise 5: Write & Run First Tests**
0:42 – 0:45	**Checkpoint: Verify, Commit & Debrief**

0:00 – 0:05	EXERCISE 1: Project Initialization with /init

Exercise 1: Project Initialization with /init

In this exercise, you'll create a fresh Python project directory and use Claude Code's /init command to bootstrap it. The /init command scans your project files, detects the tech stack, and generates a starter CLAUDE.md.

Step 1.1: Create Your Lab Directory

```
# Navigate to the labs directory
cd ~/ti-vibe-coding/labs/lab1_scaffold

# Create your personal workspace
mkdir my-sensor-toolkit && cd my-sensor-toolkit

# Initialize git
git init
echo '# Sensor Toolkit' > README.md
echo '__pycache__/' > .gitignore
echo '*.pyc' >> .gitignore
echo '.venv/' >> .gitignore
git add . && git commit -m "Initial commit"
```

Step 1.2: Run /init

```
# Start Claude Code in your new project
claude

# Inside the Claude Code session, type:
> /init

# Claude will:
#   1. Scan the directory (README.md, .gitignore)
#   2. Detect: Git repo, no Python config yet
#   3. Generate a starter CLAUDE.md
#   4. Ask you to review before saving
```

Step 1.3: Review the Generated CLAUDE.md

Claude's /init creates a starter file based on what it finds. Since this is a fresh project, the generated CLAUDE.md will be minimal. That's expected — you'll customize it in Exercise 2.

> ☆ **KEY POINT:** /init is a starting point, not a finished product. The generated CLAUDE.md captures obvious patterns but misses your team's specific conventions. Always review and refine it. Over time, use the # shortcut to add instructions you find yourself repeating.

0:05 – 0:12	EXERCISE 2: Customize CLAUDE.md for Your Team

Exercise 2: Customize CLAUDE.md for Your Team

Now you'll transform the generic /init output into a TI-specific project context file. A well-crafted CLAUDE.md dramatically improves Claude's code quality and consistency.

Step 2.1: Tell Claude Code to Enhance CLAUDE.md

Instead of editing the file manually, ask Claude Code to do it:

```
# In your Claude Code session:
> Update CLAUDE.md with the following project context:
>
> Project: TI Sensor Data Toolkit
> Purpose: Python library for validating, cleaning, and
>   analyzing semiconductor sensor data (temperature,
>   pressure, humidity readings from production lines)
> Python version: 3.10+
> Testing: pytest with fixtures in tests/conftest.py
> Style: Google Python Style Guide
> Type hints: Required on all public functions
> Docstrings: Google-style, required on all public functions
> Line length: 100 characters
> Linting: ruff
> Formatting: black
> Error handling: Never use bare except; always catch
>   specific exceptions
> Common commands:
>   pytest tests/ -v (run tests)
>   ruff check src/ (lint)
>   black src/ tests/ (format)
```

Step 2.2: Review What Claude Generated

After Claude updates CLAUDE.md, review it:

```
# Ask Claude to show it
> Show me the contents of CLAUDE.md

# Or use shell escape to read it directly
> !cat CLAUDE.md
```

Step 2.3: Add a Memory Note with

The # key in Claude Code adds quick notes to your CLAUDE.md:

```
# Press # then type a rule to add to memory:
> # Never delete or overwrite existing test files
> # Always use pathlib.Path instead of os.path
> # Sensor IDs follow format: TI-XXXX-YYYY
```

Track A (IT Engineers)	Track B (Process Engineers)
Add CI/CD workflow section: GitHub Actions, pre-commit hooks, branch naming (feature/XXXX)	Add data format section: CSV schema definitions, sensor ID patterns, expected value ranges
Add API conventions: REST endpoints, error response format, authentication patterns	Add pipeline conventions: input/output directories, logging format, report templates
Add deployment notes: Docker, staging vs. production configs	Add data quality rules: null handling, outlier thresholds, timestamp formats

☆ **KEY POINT:** Your CLAUDE.md is the single most important file for Claude Code productivity. Every instruction you add here saves you from repeating it in future prompts. Invest 5 minutes now to save hours later.

0:12 – 0:22	EXERCISE 3: Scaffold Python Project Structure

Exercise 3: Scaffold Python Project Structure

Now you'll ask Claude Code to create a production-ready Python project structure, complete with virtual environment, linting, formatting, and test configuration.

Step 3.1: Generate the Project Scaffold

```
# In your Claude Code session:
> Create a complete Python project structure for
> TI Sensor Data Toolkit with:
> - pyproject.toml with project metadata
> - src/sensor_toolkit/ package with __init__.py
> - src/sensor_toolkit/validators.py (empty module)
> - src/sensor_toolkit/cleaners.py (empty module)
> - src/sensor_toolkit/analyzers.py (empty module)
> - tests/ directory with conftest.py
> - tests/test_validators.py (placeholder)
> - .ruff.toml configured for Google style
> - Configure pytest, ruff, and black in pyproject.toml
> - requirements-dev.txt with pytest, ruff, black, pytest-cov
```

Step 3.2: Verify the Generated Structure

```
# Ask Claude to show the file tree
> Show me the complete file tree of this project

# Expected structure:
# my-sensor-toolkit/
# ├── CLAUDE.md
# ├── README.md
# ├── pyproject.toml
# ├── requirements-dev.txt
# ├── .ruff.toml
# ├── .gitignore
# ├── src/
# │   └── sensor_toolkit/
# │       ├── __init__.py
# │       ├── validators.py
# │       ├── cleaners.py
# │       └── analyzers.py
# └── tests/
#     ├── conftest.py
#     └── test_validators.py
```

Step 3.3: Set Up Virtual Environment and Install Dependencies

```
# Ask Claude to set up the dev environment:
> Create a Python virtual environment, activate it,
> and install all dev dependencies from requirements-dev.txt

# Claude will run something like:
#   python3 -m venv .venv
```

```
#   source .venv/bin/activate
#   pip install -r requirements-dev.txt

# Verify tools are available:
> !ruff --version
> !black --version
> !pytest --version
```

⚠ **WARNING:** If Claude asks permission to run bash commands (pip install, python -m venv), approve them. These are safe, expected operations. If you're unsure about any command Claude wants to run, use Shift+Tab to switch to Plan Mode and review before approving.

✅ **EXERCISE 3 CHECKPOINT**

- pyproject.toml exists with [project] and [tool.pytest] sections
- src/sensor_toolkit/ has __init__.py + three module files
- tests/ has conftest.py and at least one test file
- .ruff.toml is configured with line-length = 100
- All dev tools installed: ruff, black, pytest, pytest-cov

0:22 – 0:32	EXERCISE 4: Generate Core Modules with Claude Code

Exercise 4: Generate Core Modules with Claude Code

This is the core exercise: you'll describe what each module should do in natural language, and Claude Code will generate the implementation. This demonstrates the "vibe coding" workflow from Module 1.

Step 4.1: Generate validators.py

```
# In your Claude Code session:
> Implement src/sensor_toolkit/validators.py with:
>
> A SensorReading dataclass with fields:
>   timestamp (datetime), sensor_id (str),
>   temperature (float), pressure (float), humidity (float)
>
> Functions:
> - validate_reading(reading: SensorReading) -> list[str]
>   Returns list of validation error messages (empty = valid)
>   Rules: temp between -40..150, pressure 0..1000,
>   humidity 0..100, sensor_id matches TI-XXXX-YYYY pattern
>
> - validate_batch(readings: list[SensorReading])
>     -> dict with keys: total, valid, invalid, errors
>
> Include type hints and Google-style docstrings.
```

Step 4.2: Generate cleaners.py

```
# Continue in the same session:
> Implement src/sensor_toolkit/cleaners.py with:
>
> Functions:
> - remove_duplicates(readings: list[SensorReading])
>   Deduplicate by (timestamp, sensor_id), keep first
>
> - clamp_outliers(readings, temp_range=(-40,150),
>     pressure_range=(0,1000), humidity_range=(0,100))
>   Clamp values to valid ranges instead of removing
>
> - fill_missing_timestamps(readings, interval_seconds=60)
>   Insert placeholder readings for missing time intervals
```

Step 4.3: Generate analyzers.py (Track-Differentiated)

Track A (IT Engineers)	Track B (Process Engineers)
Generate an API-style module: calculate_statistics() returning a StatsResult dataclass with mean/median/std/min/max per sensor, detect_anomalies() using z-score thresholds, generate_report() returning a structured dict	Generate a simpler module: calculate_averages() returning dict of avg temp/pressure/humidity per sensor, find_extremes() returning min/max readings, count_by_sensor() returning reading counts per sensor_id

Add: correlate_sensors() comparing readings across sensors, group_by_time_window() for time-series bucketing	Add: summarize_daily() aggregating by date, format_csv_report() writing results to CSV string

```
# Track A prompt:
> Implement src/sensor_toolkit/analyzers.py with statistical
> analysis: calculate_statistics per sensor (mean, median,
> std, min, max), detect_anomalies using z-score > 2,
> generate_report returning structured dict.
> Use a StatsResult dataclass for return types.

# Track B prompt:
> Implement src/sensor_toolkit/analyzers.py with basic
> analysis: calculate_averages per sensor, find_extremes
> (min/max readings), count_by_sensor, summarize_daily
> aggregating by date. Keep it simple and practical.
```

☆ **KEY POINT:** Notice the workflow: you describe WHAT you want, not HOW to implement it. Claude generates the code following the standards you set in CLAUDE.md (type hints, docstrings, Google style). This is the core vibe coding loop: intent → generation → review → refine.

0:32 – 0:42	EXERCISE 5: Write & Run First Tests

Exercise 5: Write & Run First Tests

Now you'll use Claude Code to generate tests for the modules you just created. This closes the loop: code → tests → verify.

Step 5.1: Generate Tests for validators.py

```
# In your Claude Code session:
> Generate comprehensive pytest tests for
> @src/sensor_toolkit/validators.py in
> tests/test_validators.py. Include:
> - Test fixture for valid SensorReading in conftest.py
> - Happy path: valid reading returns empty error list
> - Edge cases: boundary values for each range
> - Invalid data: each validation rule individually
> - Batch validation: mixed valid/invalid readings
> - Use pytest.mark.parametrize for range boundaries
```

Step 5.2: Run the Tests

```
# Ask Claude to run the tests:
> Run the test suite with verbose output and coverage

# Claude will execute:
#   pytest tests/ -v --cov=src/sensor_toolkit --cov-report=term-missing

# If tests fail, ask Claude to fix them:
> The test_temperature_out_of_range test is failing.
> Here's the error. Fix it.

# Use /rewind if Claude's fix makes things worse:
> /rewind
```

> ☆ **KEY POINT:** The test-fix-rewind cycle is the daily workflow of vibe coding: generate tests, run them, fix failures, and use /rewind if a fix goes wrong. This is why checkpoints (Module 4) are so important — they give you a safety net for experimentation.

Step 5.3: Generate Tests for cleaners.py and analyzers.py

```
# Continue generating tests for the other modules:
> Generate pytest tests for @src/sensor_toolkit/cleaners.py
> in tests/test_cleaners.py. Test remove_duplicates with
> duplicate and unique data, clamp_outliers with extreme
> values, and fill_missing_timestamps with gaps.

# And for analyzers:
> Generate pytest tests for @src/sensor_toolkit/analyzers.py
> in tests/test_analyzers.py.
```

Step 5.4: Run Full Suite with Coverage

```
# Run everything together:
> Run all tests with coverage and show me the coverage report

# Target: 80%+ coverage on validators.py
# Acceptable: 60%+ on cleaners.py and analyzers.py

# If coverage is low, ask Claude to add more tests:
> Coverage for cleaners.py is only 55%.
> What lines are missing coverage? Add tests for them.
```

Track A: Advanced Testing	Track B: Basic Testing
Add parametrized tests with @pytest.mark.parametrize for boundary values, multiple invalid inputs, and edge cases	Focus on happy path tests and one negative test per function. Ensure all tests pass before adding complexity.
Add fixture factories and conftest.py organization with scope management	Use simple fixtures in conftest.py. One fixture for valid data, one for invalid data.
Generate property-based test ideas (fuzz testing concepts)	Run tests and fix any failures with Claude's help. Focus on understanding the test output.

0:42 – 0:45	CHECKPOINT: Verify, Commit & Debrief

Checkpoint: Verify, Commit & Debrief

Final Verification

Before committing, verify everything works together:

```
# Run the full quality pipeline:
> Run ruff check on the src directory, then run black
> to format everything, then run the full test suite
> with coverage. Show me the results.

# Or use shell escape for quick checks:
> !ruff check src/
> !black --check src/ tests/
> !pytest tests/ -v --cov=src/sensor_toolkit
```

✅ LAB 1 FINAL CHECKLIST

- CLAUDE.md exists with TI-specific project context and coding standards
- pyproject.toml configured with pytest, ruff, and black settings
- src/sensor_toolkit/ has validators.py, cleaners.py, analyzers.py with implementations
- tests/ has test files for each module with passing tests
- Coverage: 60%+ overall (80%+ on validators.py is stretch goal)
- All code passes ruff check with zero warnings
- All code formatted with black (black --check passes)

Git Commit with Claude Code

```
# Ask Claude to commit everything:
> Stage all files and create a meaningful commit message
> describing what we built: project scaffold, three core
> modules for sensor data processing, and test suite.

# Claude will generate something like:
#   feat: scaffold TI Sensor Data Toolkit with core modules
#
#   - Initialize project with pyproject.toml and dev tooling
#   - Add validators.py: SensorReading validation with range checks
#   - Add cleaners.py: deduplication, outlier clamping, gap filling
#   - Add analyzers.py: statistical analysis and anomaly detection
#   - Add pytest suite with 80%+ coverage on validators
#   - Configure ruff, black, and pytest in pyproject.toml
```

☆ **KEY POINT:** You just built a complete Python project from scratch in 40 minutes — project scaffold, three modules with real implementations, test suite with coverage, linting, and formatting. Without Claude Code, this would take 2–4 hours for an experienced developer. That's the productivity multiplier of vibe coding.

MODULE 6 RECAP

✅ Exercise 1: /init bootstraps a starter CLAUDE.md by scanning your project — always review and customize it.

✅ Exercise 2: A well-crafted CLAUDE.md is your #1 investment for Claude Code productivity — add team standards, conventions, and domain terminology.

✅ Exercise 3: Claude Code can scaffold an entire Python project (pyproject.toml, ruff, black, pytest) from a single natural language description.

✅ Exercise 4: The vibe coding workflow is intent → generation → review → refine. Describe WHAT, Claude implements HOW.

✅ Exercise 5: The test-fix-rewind cycle (generate tests → run → fix failures → /rewind if needed) is the daily coding loop.

✅ Checkpoint: Always lint, format, test, and commit. Claude Code can do all of this in a single prompt.

TRANSITION TO MODULE 7

You've now built a real project from scratch. In Module 7 (Coding with Claude Code: Refactor & Debug), you'll learn to improve existing code — refactoring for testability, auto-generating docstrings and type hints, debugging from tracebacks, and using checkpoints for safe experimentation. The project you built in this lab will be the starting point.